

Example

```
MyPickAndPlace.WaitForFreeTarget := true
```

SimTalk

[ReservedFor \[SimTalk\]](#)

[ReservedPlace \[SimTalk\]](#) of the *MUs*

[contentsAndReservedList \[SimTalk\]](#)

[TargetSelection \[SimTalk\]](#)

See also

[Wait for Free Target \[PickAndPlace\]](#)

[Target Selection \[drop-down list\]](#)

Store

Store [object]

Use the object *Store* for storing parts for a certain time. It usually represents a warehouse in your plant.

Image



Description

The *MUs* remain in the *Store* until you remove them, for example by using a *Method*. Type the number of storage places into a net of coordinates, i.e., into the text boxes **X-Dimension**, **Y-Dimension**, and **Z-Dimension**. The *Store* receives *MUs* as long as storage places are available within the storage area. The *part* triggers a sensor, when it enters the *Store*. The sensor then calls an **Entrance Control**, i.e., a *Method* object, that determines the storage place onto which the *Store* places the *part*. The **Entrance Control** can update the inventory list or execute any other action you define. If you do not define an **Entrance Control**, the *Store* places the *part* onto the first unoccupied storage place in the net of coordinates.

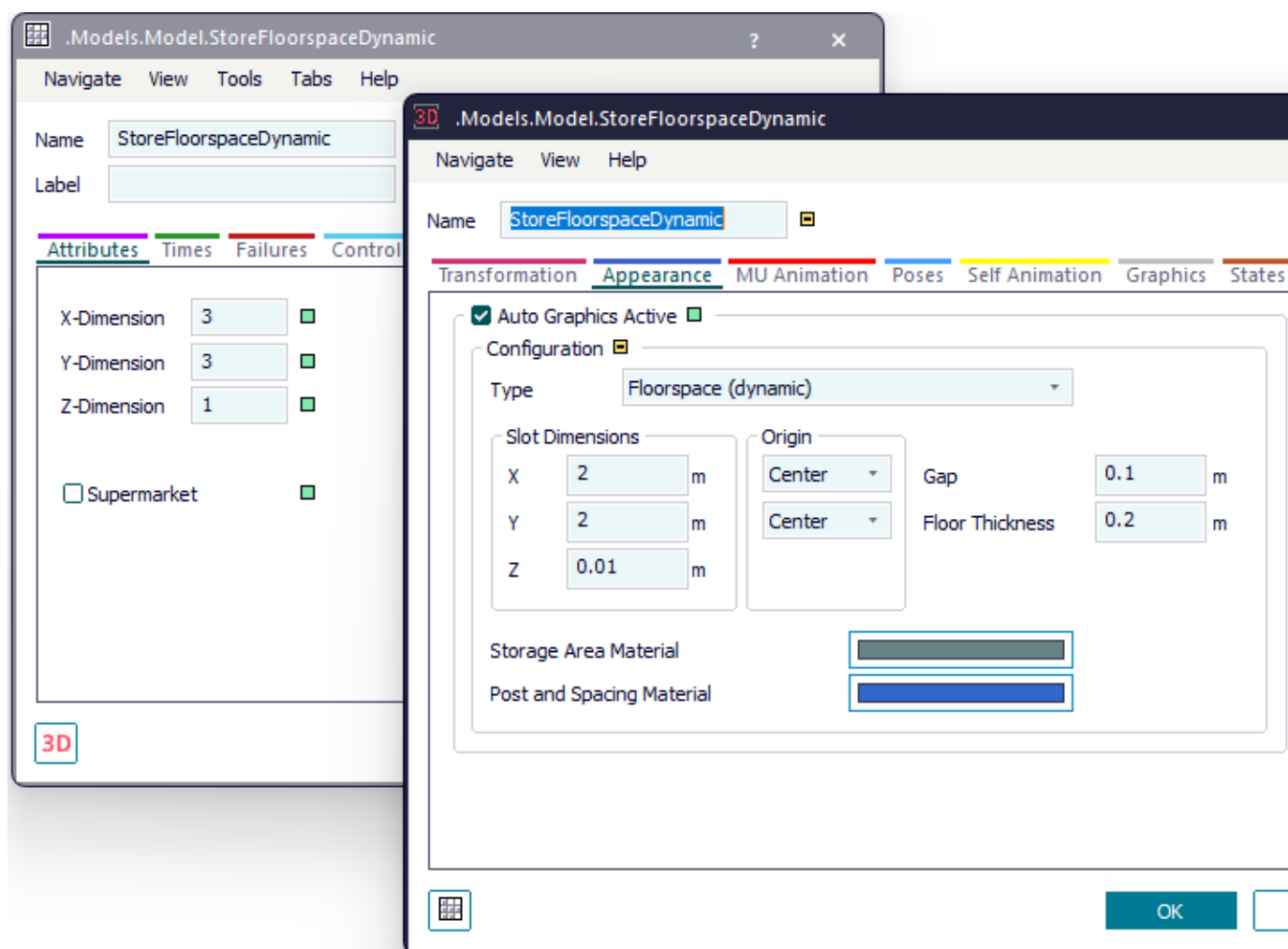
The *Store* does neither have a **Set-up Time** nor a **Processing Time**. If you decrease the size of the *Store*, you will have to delete or move *MUs* that are located outside of these new coordinates. If, for example, a *part* is located at the position (3,4), the new x-coordinate may not be less than 3, and the

new y-coordinate less than 4. During a failure the *Store* does not place any parts into storage but can still remove them from storage.

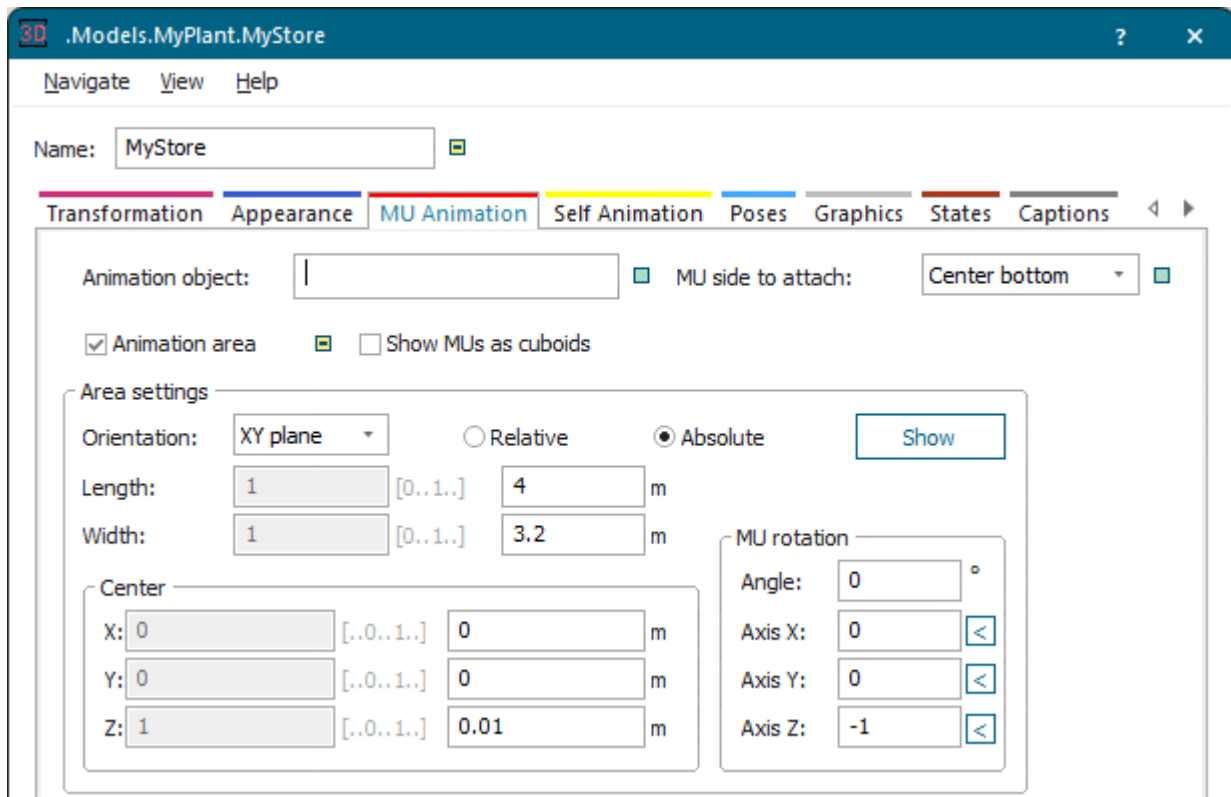
You can also configure the *Store* as a **Supermarket** in pulling material flow strategies. The *Supermarket* stores and manages the different part types and controls the fill level for each part type in the *Store*. Once the minimum stock for a part type is reached, the *Supermarket* automatically sends an order to the parts supplier so that it fills the *Store* up again.

You will typically use the *Store* to model warehouses and other storage facilities in your plant.

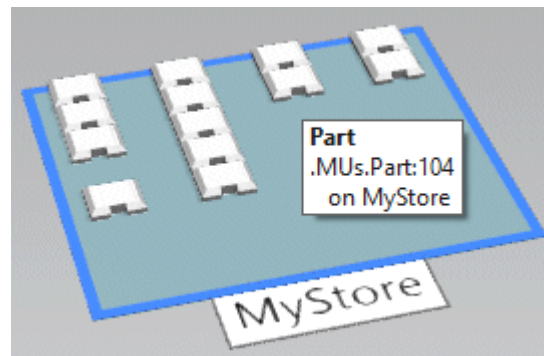
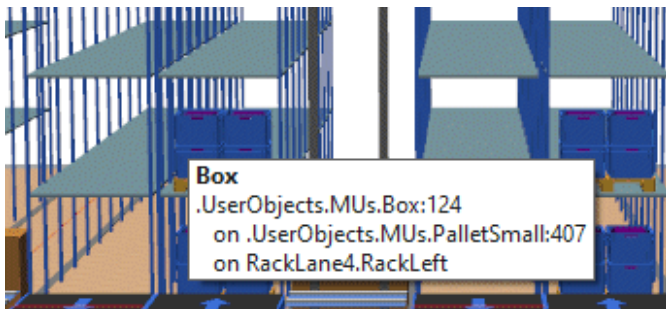
You can select different configurations for the *Store* on the tab **Appearance**.



On the tab **MU Animation** you can set how the parts are distributed across the **Animation Area** of the *Store*.



To show a **tooltip** with information about the *Store*, hover with the mouse over it.

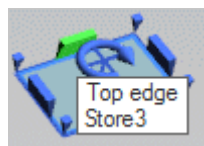


Show Manipulators

To change the length of the graphic and the anchor points of the object, click **Show Manipulators** on the **Edit** ribbon tab or press **M** on the keyboard.



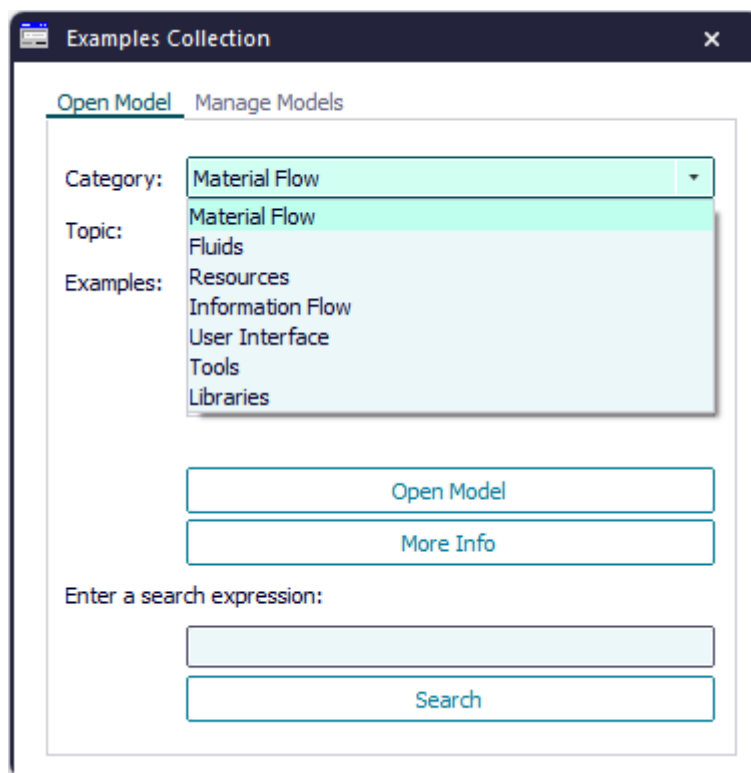
You can use the manipulators to change the dimensions of the *Store*.



Add the Object to the Simulation Model

To add the object *Store* to your simulation model, click  **Manage Class Library** > **Basic Objects** > **MaterialFlow** > **Store** on the **Home** ribbon tab.

Compare the sample models: Click the **Window** ribbon tab, click **Start Page** > **Getting Started** > **Example Models** > **Small Examples**. Then, select the respective **Category**, the **Topic**, and the **Example** in the dialog **Examples Collection**, and click **Open Model**.



See also

[Place Parts into Stock and Remove Parts from It](#)

[Use the Store as Supermarket](#)

[Stack Parts in the Store](#)

Appearance of the Store

Animation Area [described]

Video on YouTube

<https://youtu.be/ISYfZMdQp3w?si=csV--W1TIIRHgh6J>

Dialog Box of the Store

Dialog Box of the Store

Double-click the icon of the *Store* to open its dialog box.

Edit Simulation Properties

In the dialog box you can change the **simulation properties** of the object. The shared properties are described under [Dialog Items of the Objects](#).

Edit Animation Properties

To edit the **3D properties** of the object in the dialog box [Edit 3D Properties](#):

- Click the button  **Edit 3D Properties** in the lower left corner of the simulation properties dialog box.
- Select the object in the model and press the **spacebar**.

To manipulate the graphic of the object, click [Show Manipulators](#) on the **Edit** ribbon tab or press **M** on the keyboard.

Video on YouTube

<https://youtu.be/ISYfZMdQp3w?si=n9D4XzDdv5CN5Nwg&t=35>

Tab Attributes

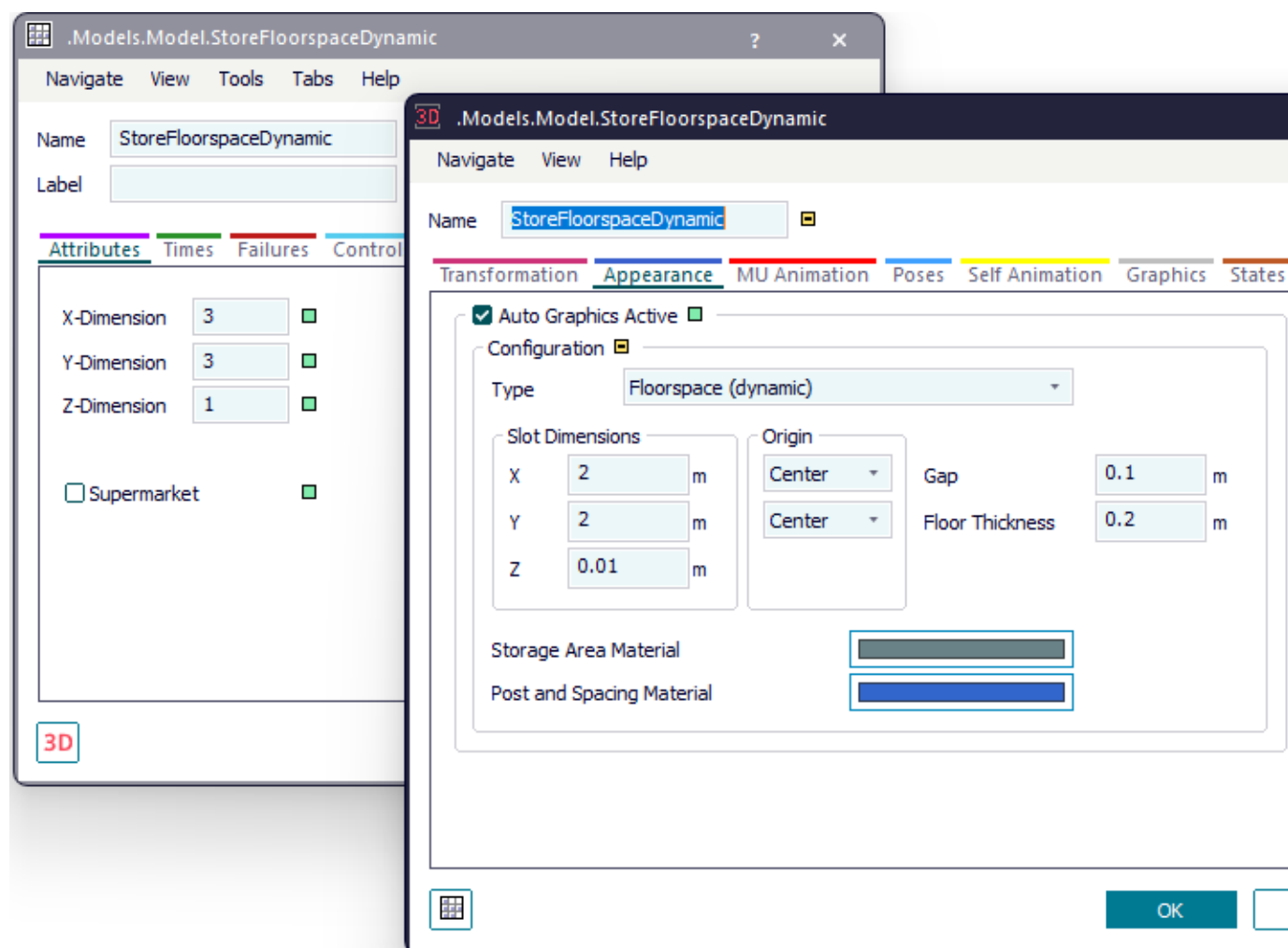
Tab Attributes [Store]

The tab **Attributes** provides the settings listed in the table of contents to the left.

The storage places of the *Store* are organized in a net of coordinates whose dimension you can set in the [X-Dimension](#), the [Y-Dimension](#), and the [Z-Dimension](#). You can access the various storage places using their coordinates.

You can edit the dimensions of the graphic of the *Store* by clicking [Show Manipulators](#) on the **Edit** ribbon tab and selecting and dragging a manipulator.

You can select different configurations for the Store on the tab **Appearance**.



Select the check box to use the Store as **Supermarket** for pulling material flow strategies. *Plant Simulation* then shows the **Configuration** button and dims the settings for the dimension.

.Models.Model.Store ? X

Navigate View Tools Tabs Help

Name: ☐ Failed ☐ Entrance Locked ☐

Label: ☐ Planned

Attributes Times Failures Controls Exit Statistics Importer Energy C ◀ ▶

X-Dimension: ☐

Y-Dimension: ☐

Z-Dimension: ☐

☒ Supermarket ☐ Configuration ☐

X-Dimension [Store]

Type the number of *MUs* which the *Store* can store along the x-axis into the text box **X-Dimension**.

X-Dimension 

Remarks

The **Capacity** of the *Store* is **X-Dimension** times **Y-Dimension** times **Z-Dimension**. The greatest allowed value is ten million.

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

SimTalk

[XDim \[SimTalk\] - Store](#)

[Capacity \[SimTalk\] - Store](#)

[pe\(X,Y\) / \[X,Y\] \[SimTalk\] - Store](#)

[setDim \[SimTalk\]](#)

See also

[Y-Dimension \[Store\]](#)

[Z-Dimension \[Store\]](#)

Y-Dimension [Store]

Type the number of *MUs* which the *Store* can store along the y-axis into the text box **Y-Dimension**.

Y-Dimension: 

Remarks

The **Capacity** of the *Store* is **X-Dimension** times **Y-Dimension** times **Z-Dimension**. The greatest allowed value is ten million.

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

SimTalk

[YDim \[SimTalk\] - Store](#)

[Capacity \[SimTalk\] - Store](#)

[pe\(X,Y\) / \[X,Y\] \[SimTalk\] - Store](#)

[setDim \[SimTalk\]](#)

See also

[X-Dimension \[Store\]](#)

[Z-Dimension \[Store\]](#)

Z-Dimension [Store]

Type the number of *MUs* which the *Store* can store along the z-axis into the text box **Z-Dimension**.

Z-Dimension: 

Remarks

The **Z-Dimension** enables to stack parts in the *Store*, for example upward in a high bay warehouse. The **Capacity** of the *Store* is **X-Dimension** times **Y-Dimension** times **Z-Dimension**

If you decrease the dimension of the *Store*, make sure that no *MUs* are located on the storage places that will be deleted by this action! Either delete these *MUs* or move them to another storage place on the smaller storage space.

SimTalk

[ZDim \[SimTalk\] - Store](#)

[getStackHeight \[SimTalk\] - Store](#)

See also

[Stack Parts in the Store](#)

[Unload Stacked Parts](#)

[X-Dimension \[Store\]](#)

[Y-Dimension \[Store\]](#)

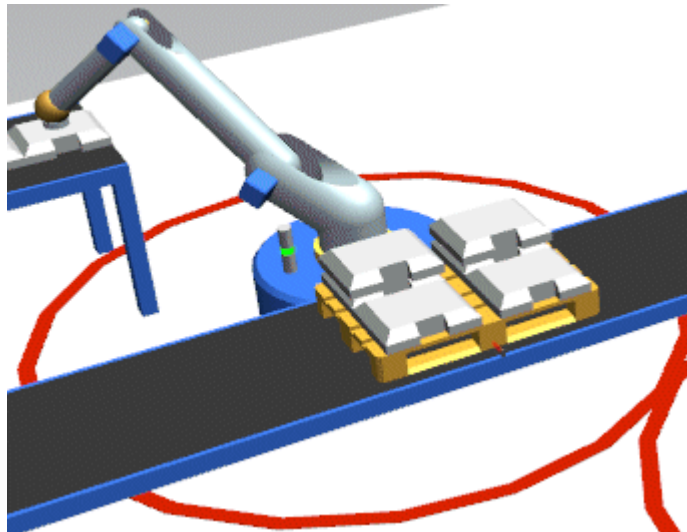
Fill Whole Layer [check box] - Store

If the **Z-Dimension** of the *Store* is greater than 1, you can stock parts layer by layer by selecting **Fill Whole Layer**.

☒ Fill Whole Layer 

Remarks

Plant Simulation always starts a new layer first and only then stacks the parts on the layer below.



By default, **Fill Whole Layer** is deactivated ☐ Fill Whole Layer  and *Plant Simulation* stacks the parts on each place up to its maximum **Z-Dimension** and then starts a new layer.

Fill Whole Layer also affects the read-only attribute **Cont**. It returns the next *MU* from the stack containing the highest number of *MUs*.

SimTalk

[FillWholeLayer \[SimTalk\] - Store](#)

[Cont \[SimTalk\] - material flow objects](#)

[Z-Dimension \[Store\]](#)

Supermarket [check box]

To use the *Store* as a **Supermarket**, select the check box. You can use the **Supermarket** to model pulling material flow strategies.

☒ Supermarket



Configuration



Remarks

Note

You can only select and clear the check box **Supermarket** if the *Store* is empty.

The *Supermarket* stores and manages the different part types and controls the fill level for each part type in the *Store*. When the minimum stock for a part type is reached, the *Supermarket* automatically sends an order to the parts supplier so that it fills up the *Store* again with the respective part type.

Configuration Table

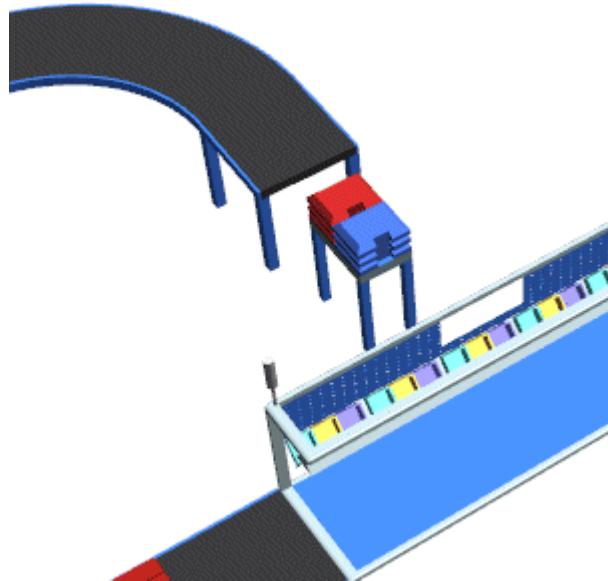
Click [Configuration](#) to open the **Configuration Table** in which you can set the **Part Type**, the **Minimum Stock**, the **Maximum Stock**, the **Initial Stock**, and the **Supplier**.

	Part Type	Name	Minimum	Maximum	Initial	Supplier	Current	Waiting
1	*.UserObjects.PartRed	PartRed	2	6	3	*.Models.Model.SourceParts	0	0
2	*.UserObjects.PartBlue	PartBlue	2	6	3	*.Models.Model.SourceParts	0	0

The **Configuration Table** also shows the **Current Stock** and the **Waiting Stock**.

Plant Simulation automatically sets the dimensions of the *Store* for the animation.

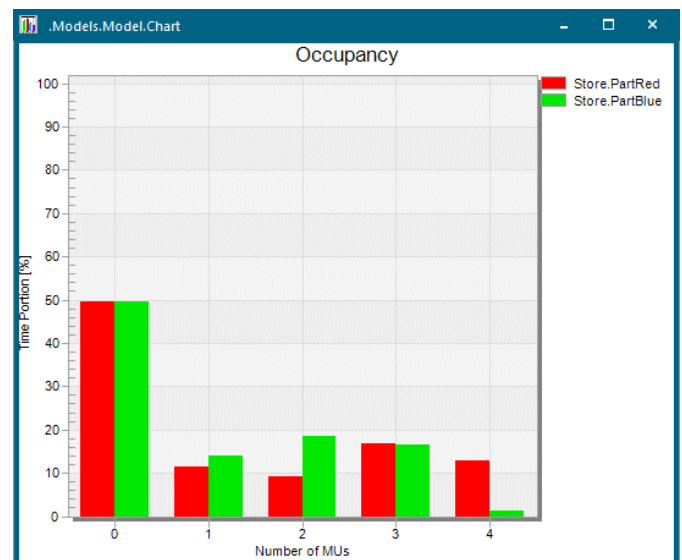
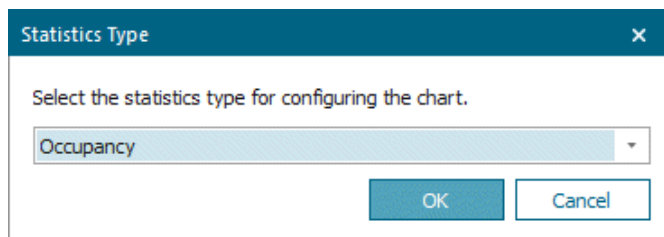
- The **X-Dimension** is always 1.
- The **Y-Dimension** shows the amount of the different part types.
- The **Z-Dimension** shows -1.
- The *Store* always places parts of the same type on a stack.



After inserting the *Store* into your model, *Plant Simulation* shows it with the default dimensions. Clicking **OK** then shows it with the settings that you typed into the **Configuration Table** and propagates the values to the child objects.

Show the Occupancy of the Supermarket in a Chart

If you drag the *Store* configured as a **Supermarket** onto a *Chart*, it shows the **occupancy** for all parts that are defined in the **Configuration Table** by default.



SimTalk

Supermarket [SimTalk]

[orderParts \[SimTalk\] - Store](#)

See also

[Use the Store as Supermarket](#)

[MU selection \[drop-down list\] > Order Controlled \[MU selection\]](#)

[Configuration \[button\]](#)

View > [Show Orders \[Source\]](#)

Video on YouTube

https://youtu.be/ISYfZMdQp3w?si=oRZapujKzN7zp5-_&t=381

Configuration [button]

To open the configuration table, click the button. Specify the **Part Type**, the **Name** of the part, the **Minimum Stock**, the **Maximum Stock**, the **Initial Stock**, and the **Supplier**



Remarks

When you activate the check box **Supermarket**, *Plant Simulation* activates the **Configuration** button.

Note

Before you can type in data, click the **Inheritance** check box so that it looks like this .

Stock – .Models.Model.Store								
	Part Type	Name	Minimum	Maximum	Initial	Supplier	Current	Waiting
1	*.UserObjects.PartRed	PartRed	2	6	3	SourceParts	0	4
2	*.UserObjects.PartBlue	PartBlue	2	6	3	SourceParts	1	5

Proceed as follows:

- Type in the **absolute path** of the **Part Type** or drag the *MU* class into the cell and drop it there.
- Type in the **Name** of the **Part Type**. If you do not specify anything, *Plant Simulation* uses the name of the object from the column **Part Type**. The names that you enter have to be unique.
- Type in the **Minimum** stock of the part type. This is the smallest amount of parts of the designated part type in the *Supermarket*.
- Type in the **Maximum** stock of the part type. This is the greatest amount of parts of the designated part type in the *Supermarket*.
- Type in the **Initial** stock of the part type. This is the amount of parts of the designated part type which are created while initializing the *Store*.

The **Initial Value** can also be smaller than the **Minimum Value**. In this case the order is triggered on **Init**.

- Type in the **Supplier** of the part type. The supplier can be another *Store*, a *Source*, or a *Method*.

When re-ordering parts, *Plant Simulation* fills the *Store* to the **Maximum** stock.

If parts leave the *Store* while an order is open, *Plant Simulation* only creates an additional order if the amount of the already ordered parts plus the current parts do not secure the **Minimum Stock**.

Specify a *Method* to set from which supplier the parts are ordered if several suppliers can provide this part.

The *Method* has to have the following signature:

```
param partName:string, minStock:integer, maxStock:integer,
currentStock:integer, orderedParts:integer
```

- The parameters contain the data from the configuration of the part which is running out of stock.
- The caller (?) of the method is the *Store*, and the active element (@) is the part which is running out of stock.
- The column **Current** shows how many parts of this part type are located in the *Store* at the moment. You cannot edit this column.
- The column **Waiting** shows the amount of remaining ordered parts. You cannot edit this column. The method *orderParts* of *Source* and *Store* increase this counter.

Click the right mouse button and select **Append Row** or **Insert Row** to add additional products and their data. Click **Delete Row** to delete the selected row.

SimTalk

[orderParts \[SimTalk\] - Store](#)

[getSupermarketConfiguration \[SimTalk\]](#)

[setSupermarketConfiguration \[SimTalk\]](#)

[Stock.MyPartName \[SimTalk\]](#)

See also

[Supermarket \[check box\]](#)

[MU selection \[drop-down list\] > Order Controlled \[MU selection\]](#)

[orderParts \[SimTalk\] - Source](#)

[View > Show Orders \[Source\]](#)

[Use the Store as Supermarket](#)

Video on YouTube

<https://youtu.be/ISYfZMdQp3w?si=oRZapujKzN7zp5-&t=381>

Tab Times

Define **Times** as described under the **Tab Times**.

Select a distribution from the drop-down list and type the values that this distribution requires into the text box. *Plant Simulation* shows the parameters along the upper border of the tab. You can also select a constant time (**Const**).

You can set the type of the distribution and a complete set of parameters with the method [setTypeAndAttr \[SimTalk\]](#).

See also

[Tab Times \[general description\]](#)

[Recovery Time \[general description\]](#)

[Recovery Time Starts](#)

[Cycle Time \[general description\]](#)

Tab Failures

Define **failures** as described under the [Tab Failures](#).

Tab Controls

Provides controls to modify the built-in behavior of the object.

Select the Path to an Existing Method

Click the ellipsis button. Navigate to the location of the *Method* in the dialog [Select Object \[for controls\]](#) and click **OK**. This inserts the name of the *Method* into the text box of the **Control**.



Press **F2** in the text box to open the *Method*. Then type in the source code of the **Control**.

Instead of choosing **Select Object**, you can also select the *Method* in a *Frame*, drag it to the text box and drop it there.

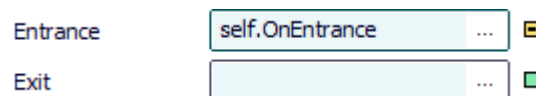
Create a Control That is a Method of the Object

Proceed as follows to create a control as a *user-defined attribute* of data type *Method*:

- Type a meaningful name into the text box and select [Create Control \[context menu\]](#). *Plant Simulation* then inserts `self.Name_you_typed_in_for_the_control`, such as `self.A1Ctrl`.



- Select **Create Control** on the empty text box. *Plant Simulation* then inserts `self.OnBuilt_in_name_of_the_control`, such as `self.OnEntrance`.



Type the source code of this control into the *Method* that opens.

To edit the source code later on:

- Press **F2**.
- Or hold down **Shift** and double-click into the text box.
- Or select **Open Object** on the context menu.
- Or click the tab **User-defined** and double-click the name of the *Method* in the list.

- To delete this control, delete the *user-defined attribute*. If you only delete the name from the text box, the *user-defined attribute* is retained.

See also

[Entrance Control \[general description\]](#)

[Exit Control \[general description\]](#)

[Pull Control \[general description\]](#)

[Shift Calendar \[tab Controls\]](#)

Tab Exit

Select to which of its successors the object moves the *MU* on the **Tab Exit**.

See also

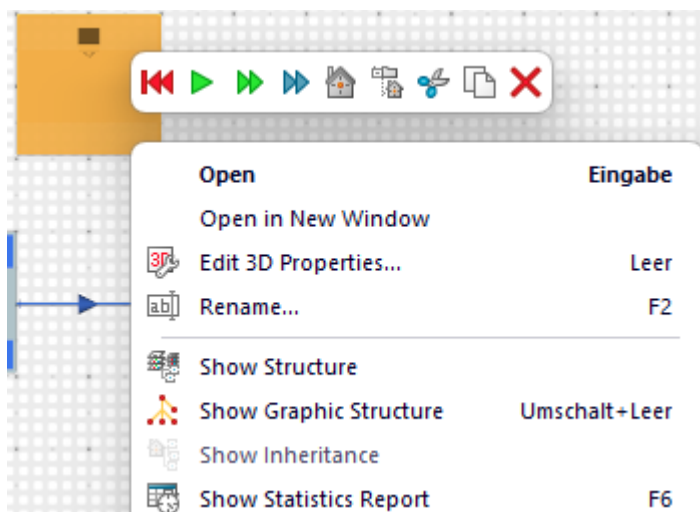
Blocking [exit strategy]

Strategy [material flow objects]

Tab Statistics

Statistics is described under the [Tab Statistics](#).

To view **Resource Statistics of Stationary Resources** in the **Statistics Report**, select **View > Show Statistics Report** in the dialog of the object. You can also click the right mouse button in the *Frame* and select **Show Statistics Report** or you can press **F6**.



See also

[Resource Statistics \[check box\]](#)

[Resource Type](#)

Tab Energy

Select **energy settings** for the object on the [Tab Energy](#).

Tab Costs

Select **costs settings** for the object on the [Tab Costs](#).

While the *Store* places parts into storage costs accrue which result from the sum of the investment costs and the operating costs.

Note

The investment costs only accrue during the **Depreciation Period**.

The costs are allocated to the part, proportional to the capacity, as **accrued costs**.

If the *Store* is empty, the costs remain with the *Store* as **general costs**.

See also

[Simulate the Accrued Costs of the Machines](#)

[CostAnalyzer > How the CostAnalyzer Assigns Costs to Part Types](#)

[CostAnalyzer > Costs Shown in the Costs Report](#)

Tab User-defined

Define your own attributes as described under the [Tab User-defined](#).

Navigate Menu

The commands are described under the [Navigate Menu](#).

View Menu

View Menu

The **View Menu** provides commands to access its functions.

Refresh [on View menu]	Show Orders [Source]
Show Statistics Report [on View menu]	Contents [material flow objects]
Show Attributes and Methods [on View menu]	Forward Blocking List

See also

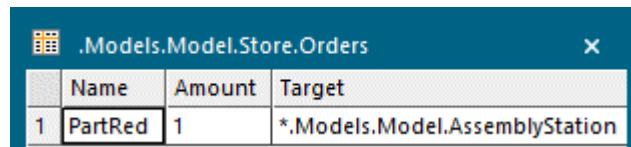
[View Menu \[general description\]](#)

Show Orders [Store]

Opens a table that shows the orders which are pending with the *Store*.

Remarks

The table contains the **Part Type** of the ordered *MU*, its **Amount**, and the **Target** of the part. The **Target** is the object that orders the parts.



	Name	Amount	Target
1	PartRed	1	*.Models.Model.AssemblyStation

See also

[Supermarket \[check box\]](#)

[Configuration \[button\]](#)

[MU selection \[drop-down list\]](#) > [Order Controlled \[MU selection\]](#)

Exit Blocking List [Store]

Opens a list, which contains all *MUs*, which are waiting to be carried away by a *Worker*.

Remarks

The table **Waiting for Importers** shows the path of the *MU*, its folder, its name, and its number, which waits for a *Worker* to pick it up and carry it away.



Waiting for Importers	
Object	
1	*UserObjects.PartA:19

SimTalk

[exitBlockList \[SimTalk\] - material flow objects](#)

[exitBlockList \[SimTalk\] - lane A or B](#)

See also

[Exit Blocking List](#)

[Exit Locked \[material flow objects\]](#)

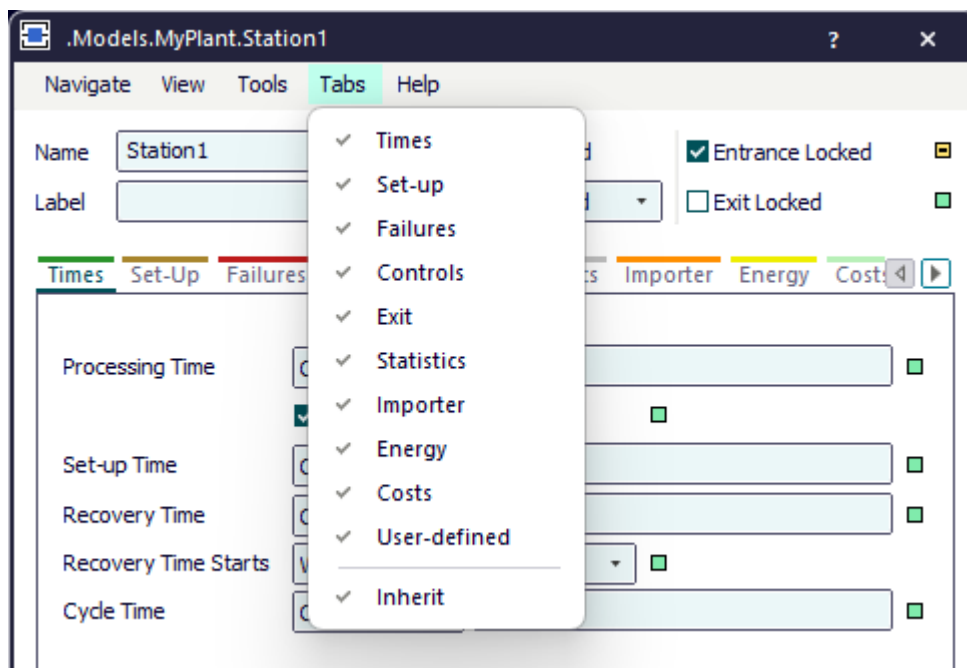
[Check the Contents List of the Stations](#)

Tools Menu

The commands are described under the [Tools Menu](#).

Tabs Menu

Use the commands of the **Tabs** menu to show or hide individual tabs of the selected material flow objects. If you hide tabs that you do not need, *Plant Simulation* opens the dialog faster, and you can change to those tabs faster that you need in your daily work.



Remarks

To apply the changed settings, click **OK**, close the dialog, and reopen it.

The menu shows a check mark to the left of the displayed tabs.

The command **Inherit** turns inheritance of the displayed or hidden tabs in the dialog off or on.

Help Menu

The commands are described under the [Help Menu](#).

Methods of the Store

_Methods of the Store

The Store provides:

- The *methods* listed in the table of contents to the left.
- The [Methods of the Material Flow Objects](#).
- The [Methods of All Objects](#).

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window [Show Attributes and Methods](#). The figure below illustrates the information using the example of the object *Station*.

The figure shows two screenshots of the 'Show Attributes and Methods' window for the 'Station' object. The first screenshot shows the 'deleteAttr' method, and the second screenshot shows the 'MUPart' method. Both screenshots have red boxes highlighting the method name, the parameter identifier and data type, and the return value data type.

First Screenshot: .Models.Model.Station

Name	Value	Inherited	Watchable	Signature
contentsList	[]			([Contents:table]) -> any
createAttr				(NameOfUserDefinedAttribute:string, DataType:string) -> boolean
createIcon				([IconName:string, Width:integer, Height:integer]) -> integer
deleteAttr				(NameOfUserDefinedAttribute:string) -> boolean

Name of the method: deleteAttr
 Identifier and data type of the parameter you enter: NameOfUserDefinedAttribute:string
 Data type of the return value: boolean

Second Screenshot: .Models.Model.Station

Name	Value	Inherited	Watchable	Signature
moveToFolder				(Folder:object) -> object
MU	VOID			([Index:integer:=1]) -> object
MUPart	VOID			([Index:integer:=1]) -> object

Name of the method: MUPart
 Identifier and data type of the parameter you enter: [Index:integer:=1]
 Default value: 1
 Data type of the return value: object

- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected [Class \[general description\]](#).
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected [Instance \[general description\]](#).

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

Note

Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

If the *method* has a return value, the signature shows its data type after the arrow `->`, `→ boolean` in the example above.

findFreePlace [SimTalk]

Finds a free storage place in the *Store* designated by <Path> and returns it.

Remarks

The method applies to the *Store*, the *ParallelStation*, the *Container*, and to the *Transporter* with a loading space of type **Store**.

Type

Method

Syntax

```
<Path>.findFreePlace([StartingAtEnd:boolean:=false, XStart:integer:=1,  
YStart:integer:=1]) → any
```

Parameters

You can specify the following parameters:

- The optional parameter *StartingAtEnd* of data type *boolean* specifies if the search is to proceed starting at the end backward toward the front (*true*) or not (*false*).

Default Value of the Parameter

The default value is *false*.

- The optional parameter *XStart* of data type *integer* designates the X-coordinate of the storage place from which on the search is to be started.

Default Value of the Parameter

The default value is 1.

- The optional parameter *YStart* of data type *integer* designates the Y-coordinate of the storage place from which on the search is to be started.

Default Value of the Parameter

The default value is 1.

Return Value

The return value has the data type *any*.

Example

```
var freePlace := .MUs.Container:1.findFreePlace
if freePlace /= void
  var x,y:integer

  freeplace.getStoragePlace(x, y)

  print "Found free place at (", x, ", ", y, ")"
end
```

See also

[findFreePlaceInRange \[SimTalk\]](#)

findFreePlaceInRange [SimTalk]

Finds a free storage place within the designated range of the *Store* designated by <Path> and returns the position of the free place.

Type

Method

Syntax

```
<Path>.findFreePlaceInRange(SearchRange:listrange[,  
LeftToRight:boolean:=true, TopToBottom:boolean:=true]) -> any
```

Parameters

You can specify the following parameters:

- The parameter *SearchRange* of data type *listrange* designates the range of the list to be searched.
- The optional parameter *LeftToRight* of data type *boolean* sets if the list is searched from left to right (*true*) or from right to left (*false*).

Default Value of the Parameter

The default value is *true*.

- The optional parameter *TopToBottom* of data type *boolean* sets if the list is searched from top to bottom (*true*) or from bottom to top (*false*).

Default Value of the Parameter

The default value is *true*.

Return Value

The return value has the data type *any*.

Example

```
MyStore.findFreePlaceInRange({2,2}..{*. *}, false)
```

See also

[findFreePlace \[SimTalk\]](#)

findPart [SimTalk] - Store

Finds the part with the specified name in the *Store* designated by <Path> and returns it.

Type

Method

Syntax

```
<Path>.findPart(PartType:string) → object
```

Parameter

The parameter *PartType* of data type *string* designates the part type.

Return Value

The return value has the data type *object*.

Example

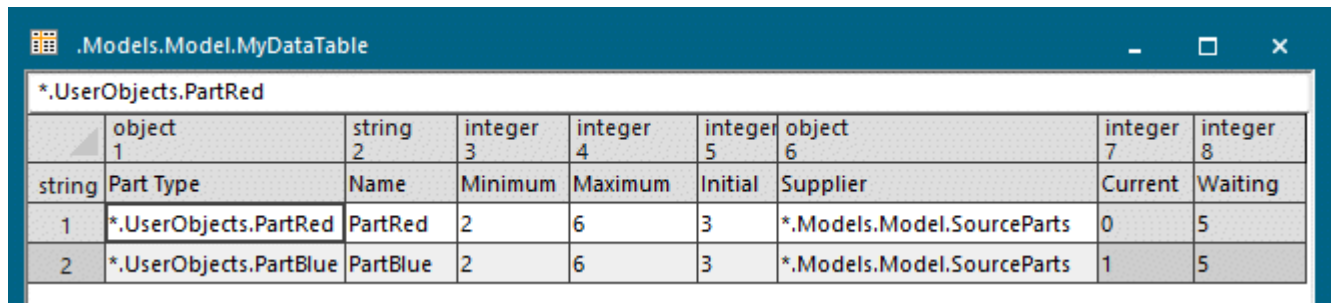
```
var o: object := MyStore.findPart("Container")  
// returns for example .MUs.Container:1
```

getSupermarketConfiguration [SimTalk]

Returns the configuration table of the *Store* designated by <Path>.

Remarks

The configuration table might look like this:



	object 1	string 2	integer 3	integer 4	integer 5	object 6	integer 7	integer 8
string	Part Type	Name	Minimum	Maximum	Initial	Supplier	Current	Waiting
1	*.UserObjects.PartRed	PartRed	2	6	3	*.Models.Model.SourceParts	0	5
2	*.UserObjects.PartBlue	PartBlue	2	6	3	*.Models.Model.SourceParts	1	5

Type

Method

Syntax

```
<Path>.getSupermarketConfiguration(Target:table)
```

Parameter

The parameter *Target* of data type *table* designates the *DataTable* into which the configuration settings will be written.

Example

```
MyStore.getSupermarketConfiguration(MyDataTable)
```

SimTalk

setSupermarketConfiguration [SimTalk]

See also

Configuration [button]

orderParts [SimTalk] - Store

Orders parts from the *Store* designated by <Path>.

Type

Method

Syntax

```
<Path>.orderParts(PartType:string, Amount:integer, Target:object)
```

Parameters

You can specify the following parameters:

- The parameter *PartType* of data type *string* designates the name of the part type that is going to be ordered.
- The parameter *Amount* of data type *integer* designates the amount of parts that is going to be ordered.
- The parameter *Target* of data type *object* designates the object that orders the parts. The target, i.e., the destination object, can be any material flow object.

If the *Target* is a **Supermarket**, *Plant Simulation* increases the counter of the amount of remaining ordered parts. Compare the column **Waiting** in the **Configuration table** of the *Store*.

Examples

```
MyStore.orderParts("MyPart", 12, MyStation)
```

```
param partsName:string, minStock:integer, maxStock:integer,
currentStock:integer, orderedParts:integer

var amount:integer := maxStock-currentStock-orderedParts
if amount > 0 then
    MyStore.orderParts(partsName, amount, ?)
end
```

SimTalk

[orderParts \[SimTalk\] - Source](#)

See also

[Configuration \[button\]](#)

Supermarket ☐

pe(X,Y) / [X,Y] [SimTalk] - Store

Sets the designated storage place on the production element (*PE*) in the *Store* designated by <Path>.

Remarks

Instead of *pe([X,Y])* you can also use *[X,Y]*.

To access the *MU* that is located on the designated storage place, append *.Cont*.

Note

The read-only attribute *Cont* returns the *MU* at the called position.

Type

Method

Syntax

```
<Path>.pe([X:integer, Y:integer]) -> any
<Path>[X:integer, Y:integer] -> any
```

Parameters

You can specify the following parameters:

- The optional parameter *X* of data type *integer* designates the **X-Dimension** of the storage place.
- The optional parameter *Y* of data type *integer* designates the **Y-Dimension**.

If you do not specify the parameters, *Plant Simulation* returns the first free *PE* or the *PE* on the place (1,1) if no free *PE* is available.

Return Value

The return value has the data type *any*.

Example

```
@.move(MyStore.pe(2,3))
MyStore[2,3].Cont.move(Station)
```


SimTalk

[Cont \[SimTalk\] - material flow objects](#)

See also

[_Methods and Read-Only Attributes of the Place in the Store \[PE\]](#)

[X-Dimension \[Store\]](#)

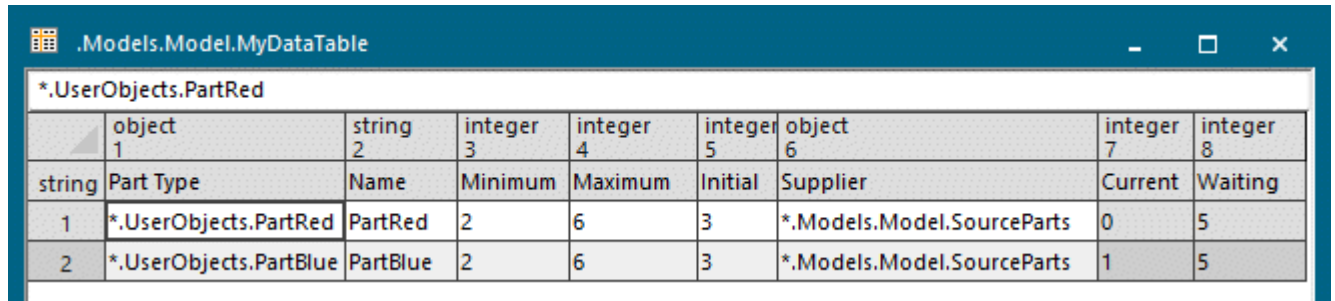
[Y-Dimension \[Store\]](#)

setSupermarketConfiguration [SimTalk]

Sets the configuration table of the *Store* designated by <Path>.

Remarks

The configuration table might look like this:



	object 1	string 2	integer 3	integer 4	integer 5	object 6	integer 7	integer 8
string	Part Type	Name	Minimum	Maximum	Initial	Supplier	Current	Waiting
1	*.UserObjects.PartRed	PartRed	2	6	3	*.Models.Model.SourceParts	0	5
2	*.UserObjects.PartBlue	PartBlue	2	6	3	*.Models.Model.SourceParts	1	5

Type

Method

Syntax

```
<Path>.setSupermarketConfiguration(Source:table/void)
```

Parameter

The parameter *Source* of data type *table* designates the *DataTable* which contains the configuration. When setting the configuration *Plant Simulation* ignores the columns **Current** and **Waiting**.

Specify *void* for the parameter *Source* to activate inheritance of the **Configuration** table of the *Store*.

Examples

```
MyStore.setSupermarketConfiguration(MyDataTable)
```

```
MyStore.setSupermarketConfiguration(void) -- activates inheritance of the  
configuration table
```

SimTalk

getSupermarketConfiguration [SimTalk]

See also

[Configuration \[button\]](#)

Methods and Read-Only Attributes of the pe

_Methods and Read-Only Attributes of the Place in the Store [PE]

The *PE* (**production element**), i.e., the place in the *Store*, provides the *methods* and *read-only attributes* listed in the table of contents to the left.